Database Mapping Document
&
System Design Justification

Multi-Vendor E-Commerce Platform

---

1. Overview

This document describes the relational database schema for the multi-vendor e-commerce platform. It covers each table's columns, data types, constraints, and relationships, along with justification for key design decisions.

The database is designed for SQL Server with Entity Framework Core as the ORM. All primary keys use UNIQUEIDENTIFIER (GUID) to avoid leaking sequential IDs in public APIs.

2. Entity Summary

Entity	Purpose
Merchant	A registered seller account. Owns products and has associated refresh tokens.
RefreshToken	Stores long-lived tokens used to issue new JWT access tokens.
Product	A product listed by a merchant. Has attributes, variants, and images.
ProductAttribute	A named attribute type for a product (e.g. Color, RAM, Size).
AttributeOption	A valid value for a product attribute (e.g. Red, 8GB, Medium).
ProductVariant	A specific purchasable combination of attribute options for a product.
VariantAttributeValue	Pivot table linking a variant to its chosen attribute options.
ProductImage	An image associated with a product, with ordering and primary flag.

3. Table Definitions

3.1 Merchant

Represents a registered seller on the platform. Authentication credentials are stored here. The merchant entity is the root of data ownership — all products and tokens are associated with a merchant.

Column	Data Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	Unique identifier for the merchant
Name	NVARCHAR(100)	NOT NULL	Display name of the merchant
Email	NVARCHAR(255)	NOT NULL, UNIQUE	Login email, must be unique system-wide
Password	NVARCHAR(255)	NOT NULL	Bcrypt hashed password, never stored as plaintext
CreatedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()	Account creation timestamp (UTC)

3.2 RefreshToken

Stores refresh tokens issued during login. Each token is associated with one merchant and can be revoked individually (logout) or in bulk. Token rotation is implemented by revoking the old token and issuing a new one on each refresh request.

Column	Data Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	Unique token identifier
MerchantId	UNIQUEIDENTIFIER	FK → Merchant.Id, NOT NULL	Owner of this refresh token
Token	NVARCHAR(500)	NOT NULL, UNIQUE	Cryptographically random token string
ExpiresAt	DATETIME2	NOT NULL	Expiry timestamp — tokens are rejected after this
IsRevoked	BIT	NOT NULL, DEFAULT 0	Set to 1 on logout or token rotation
CreatedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()	Token issuance timestamp (UTC)

3.3 Product

A product represents an item listed by a merchant. Products support soft deletion via IsDeleted and DeletedAt — records are never hard deleted to preserve historical integrity. The Status field controls visibility (draft, active, archived).

Column	Data Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	Unique product identifier
MerchantId	UNIQUEIDENTIFIER	FK → Merchant.Id, NOT NULL	Owning merchant — enforces data isolation
Name	NVARCHAR(200)	NOT NULL	Product display name
Description	NVARCHAR(MAX)	NULL	Full product description, supports long text
Status	NVARCHAR(20)	NOT NULL, DEFAULT 'draft'	Enum: draft | active | archived
BasePrice	DECIMAL(18,2)	NOT NULL, ≥ 0	Default price; variants may override this
IsDeleted	BIT	NOT NULL, DEFAULT 0	Soft delete flag — never hard delete products
DeletedAt	DATETIME2	NULL	Timestamp of soft deletion
CreatedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()	Record creation timestamp (UTC)
UpdatedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()	Last modification timestamp (UTC)

3.4 ProductAttribute

Defines the attribute types available for a product. For example, a T-Shirt product might define 'Color' and 'Size' attributes, while a Laptop defines 'RAM' and 'Storage'. Attributes are product-specific and not shared across products.

Column	Data Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	Unique attribute identifier
ProductId	UNIQUEIDENTIFIER	FK → Product.Id, NOT NULL	The product this attribute belongs to
Name	NVARCHAR(100)	NOT NULL	Attribute label, e.g. 'Color', 'RAM', 'Size'

3.5 AttributeOption

Defines the allowed values for a given ProductAttribute. For example, the 'Color' attribute might have options 'Red', 'Blue', and 'Green'. Options are constrained to their parent attribute, preventing cross-product contamination.

Column	Data Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	Unique option identifier
ProductAttributeId	UNIQUEIDENTIFIER	FK → ProductAttribute.Id, NOT NULL	The attribute this option belongs to
Value	NVARCHAR(100)	NOT NULL	Option value, e.g. 'Red', '8GB', 'Small'

3.6 ProductVariant

Represents a specific purchasable combination of attribute options for a product (e.g. 'Red / Small' T-shirt). Each variant has its own SKU, stock quantity, and optional price override. The SKU column has a unique index enforced at the database level.

Column	Data Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	Unique variant identifier
ProductId	UNIQUEIDENTIFIER	FK → Product.Id, NOT NULL	The product this variant belongs to
SKU	NVARCHAR(100)	NOT NULL, UNIQUE	Stock Keeping Unit — globally unique across all variants
Quantity	INT	NOT NULL, ≥ 0	Current stock quantity
LowStockThreshold	INT	NULL, ≥ 0	Quantity at which a low-stock alert is triggered
PriceOverride	DECIMAL(18,2)	NULL, ≥ 0	If set, overrides Product.BasePrice for this variant
CompareAtPrice	DECIMAL(18,2)	NULL, ≥ 0	Original price shown for strikethrough/sale display
DiscountStartDate	DATETIME2	NULL	Start of discount period (UTC)
DiscountEndDate	DATETIME2	NULL	End of discount period (UTC)
IsActive	BIT	NOT NULL, DEFAULT 1	Whether this variant is available for purchase

Column	Data Type	Constraints	Description
IsDeleted	BIT	NOT NULL, DEFAULT 0	Soft delete flag
DeletedAt	DATETIME2	NULL	Timestamp of soft deletion
CreatedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()	Record creation timestamp (UTC)
UpdatedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()	Last modification timestamp (UTC)

3.7 VariantAttributeValue

A pure pivot table that links a ProductVariant to its chosen AttributeOptions. A 'Red / Small' variant is represented as two rows in this table — one linking to the 'Red' option and one linking to the 'Small' option. The composite primary key (VariantId, AttributeOptionId) prevents duplicate assignments.

Column	Data Type	Constraints	Description
VariantId	UNIQUEIDENTIFIER	CPK, FK → ProductVariant.Id, NOT NULL	The variant being described
AttributeOptionId	UNIQUEIDENTIFIER	CPK, FK → AttributeOption.Id, NOT NULL	The chosen option for one attribute

3.8 ProductImage

Stores image URLs for a product. Multiple images are supported with display ordering. The IsPrimary flag marks the main image used in listings and search results.

Column	Data Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PK, NOT NULL	Unique image identifier
ProductId	UNIQUEIDENTIFIER	FK → Product.Id, NOT NULL	The product this image belongs to
ImageUrl	NVARCHAR(1000)	NOT NULL	Fully qualified URL to the stored image
IsPrimary	BIT	NOT NULL, DEFAULT 0	Marks the main display image for the product

Column	Data Type	Constraints	Description
DisplayOrder	INT	NOT NULL, DEFAULT 0	Controls image sort order in galleries
CreatedAt	DATETIME2	NOT NULL, DEFAULT GETUTCDATE()	Upload timestamp (UTC)

4. Entity Relationships

The following table documents all foreign key relationships, their cardinality, and the configured delete behavior.

Relationship	Type	FK Column	Behavior
Merchant → RefreshToken	1 : M	RefreshToken.MerchantId	CASCADE DELETE — tokens removed when merchant deleted
Merchant → Product	1 : M	Product.MerchantId	RESTRICT DELETE — merchant cannot be deleted while products exist
Product → ProductAttribute	1 : M	ProductAttribute.ProductId	CASCADE DELETE — attributes removed with product
Product → ProductVariant	1 : M	ProductVariant.ProductId	CASCADE DELETE — variants removed with product
Product → ProductImage	1 : M	ProductImage.ProductId	CASCADE DELETE — images removed with product
ProductAttribute → AttributeOption	1 : M	AttributeOption.ProductAttributeId	CASCADE DELETE — options removed with attribute
ProductVariant → VariantAttributeValue	1 : M	VariantAttributeValue.VariantId	CASCADE DELETE — assignments removed with variant
AttributeOption → VariantAttributeValue	1 : M	VariantAttributeValue.AttributeOptionId	RESTRICT DELETE — cannot delete option in use by a variant

5. Index Strategy

Indexes are defined on all foreign key columns and columns used in frequent WHERE clauses. Entity Framework Core does not add indexes to foreign key columns automatically — these must be configured explicitly.

Table	Index Columns	Type	Reason
Product	MerchantId	Non-unique	Filter products by merchant on every list request
Product	Status	Non-unique	Filter by status (active/draft/archived)
Product	IsDeleted	Non-unique	Global query filter for soft delete
ProductVariant	SKU	Unique	Enforce SKU uniqueness and support fast lookup
ProductVariant	ProductId	Non-unique	List variants by product
ProductVariant	IsDeleted	Non-unique	Global query filter for soft delete
ProductAttribute	ProductId	Non-unique	Retrieve attributes for a product
AttributeOption	ProductAttributeId	Non-unique	Retrieve options for an attribute
RefreshToken	Token	Unique	Fast token lookup on every refresh request
RefreshToken	MerchantId	Non-unique	Revoke all tokens for a merchant on logout

6. Variant System Design Justification

6.1 Approach selected: EAV with pivot table

The variant system uses an Entity-Attribute-Value (EAV) pattern implemented through three tables: ProductAttribute, AttributeOption, and VariantAttributeValue. This approach was selected over the two alternatives below.

Alternative 1 — Hardcoded columns (rejected)

Adding columns like color, size, and ram directly to the ProductVariant table would require a schema migration for every new product type. This approach violates the assessment requirement of 'no hardcoded product-specific columns' and cannot support the unlimited attribute requirement.

Alternative 2 — JSON column (rejected)

Storing attributes as a JSON blob on the variant (e.g. { "color": "red", "size": "M" }) is flexible but has significant drawbacks. JSON fields cannot be filtered or indexed efficiently in standard SQL. There is no referential integrity — values like 'Redd' and 'Red' are both valid. The approach is not suitable for a platform where attribute-based filtering will be a core feature.

Selected: Relational EAV

The selected approach stores attribute definitions and their valid values as proper relational rows. This provides:

- Unlimited attributes per product — each is a row in ProductAttribute, no schema changes required
- Unlimited attribute values — each is a row in AttributeOption
- Unlimited variant combinations — each variant is linked to its options via VariantAttributeValue
- Full SQL queryability — filter variants by attribute value using standard JOINs
- Referential integrity — it is impossible to assign an invalid or cross-product attribute option to a variant
- Composite PK on VariantAttributeValue prevents a variant from having duplicate or conflicting values for the same attribute

6.2 Scalability considerations

The EAV approach adds join depth compared to a flat schema. A full variant read (with all attribute labels and values) requires joining across four tables: ProductVariant → VariantAttributeValue → AttributeOption → ProductAttribute. At the scale of this platform, this is an acceptable cost.

For high-read scenarios, the following strategies mitigate this:

- Index all foreign key columns (see Section 5)
- Cache frequently accessed product and variant data at the application layer
- Use projection queries (SELECT only needed columns) rather than loading full entity graphs

6.3 Trade-offs

Trade-off	Impact	Mitigation
Join depth for variant reads	Requires 3–4 table joins to read a full variant	Indexed FKs, application-layer caching
No constraint on attribute count per variant	A variant could technically have two values for the same attribute	Enforce uniqueness on (VariantId, AttributeId) in service layer
Attribute values are free text	No data type enforcement on option values (e.g. numeric RAM)	Acceptable for this platform scope; can add a Type column to ProductAttribute later
Schema complexity	More tables than a simple JSON or flat approach	Complexity is justified by the queryability and integrity guarantees

7. Soft Delete Strategy

Product and ProductVariant use soft deletion (IsDeleted + DeletedAt) rather than hard DELETE statements. This is standard practice in e-commerce systems for the following reasons:

- Historical integrity — deleted products may be referenced in future order history tables
- Recoverability — merchants can restore accidentally deleted items
- Audit trail — DeletedAt records exactly when the deletion occurred

A global EF Core query filter (HasQueryFilter) is applied to both Product and ProductVariant so that soft-deleted records are automatically excluded from all queries without requiring explicit WHERE clauses in application code.

8. Security Notes

- All passwords are stored as bcrypt hashes. The column is named 'Password' in the schema but stores only the hash — never plaintext.
- All primary keys are GUIDs (UNIQUEIDENTIFIER), not sequential integers, to prevent enumeration attacks via public API IDs.
- MerchantId is embedded in the JWT access token claims. All service-layer operations read the merchant identity from the token, never from the request body, preventing horizontal privilege escalation.
- Refresh tokens are single-use and rotated on every refresh request. A reused token triggers full revocation of the token family.